Movie Theater System

Software Requirements Specification

Version 1

September 24, 2026

Group 5
Adriane, Chris, Domenic

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2026

Revision History

Date	Description	Author	Comments
09/24/26	Version 1	Adriane, Chris, Dom	First Revision - includes everything up to functional and non-functional requirements
10/08/26	Version 2	Adriane, Chris, Dom	Second Revision -
10/22/26	Version 3	Adriane, Chris, Dom	Third Revision -

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Adriane Gonzales	Software Eng.	09/24/2026
	Chris Nguyen	Software Eng.	09/24/2026
	Dom Slewa	Software Eng.	09/24/2026
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces *3*
3.1.2 Hardware Interfaces *3*
3.1.3 Software Interfaces *3*
3.1.4 Communications Interfaces *3*
3.2 Functional Requirements 3
3.2.1 Find Movie and SHowtimes *3*
3.2.2 Purchase Tickets *3*
3.2.3 Manage Customer Accounts *3*
3.2.4 Manage Theater Information *3*
3.2.5 Collect Customer Feedback *3*
3.2.6 Check, Return, and Exchange Tickets *3*
3.3 Use Cases 3
3.3.1 Use Case #1 ..... *3*
3.3.2 Use Case #2 ..... *3*
3.4 Classes / Objects 3
3.4.1 Customer Account *3*
3.4.2 Showtime *3*
3.4.3 Ticket *3*
3.5 Non-Functional Requirements 4
3.5.1 Performance *4*
3.5.2 Reliability *4*
3.5.3 Availability *4*
3.5.4 Security *4*
3.5.5 Maintainability *4*
3.5.6 Portability *4*
3.6 Inverse Requirements 4
3.7 Design Constraints 4
3.8 Logical Database Requirements 4

3.9 Other Requirements....4
4. Analysis Models....4
4.1 Sequence Diagrams....5
4.3 Data Flow Diagrams (DFD)....5
4.2 State-Transition Diagrams (STD)....5
5. Change Management Process....5
A. Appendices....5
A.1 Appendix 1....5
A.2 Appendix 2....5

1. Introduction

This document describes the Movie Theater System's users, features, limits, and requirements.

1.1 Purpose

This document records the requirements from client interviews and class materials. It is for the client, our group, the instructor, and developers. It will guide the system's design and testing.

1.2 Scope

The Movie Theater System will support website and kiosk ticket sales for 20 San Diego theaters. Employees will handle sales and refunds, while administrators manage theater information. It aims to make purchases easier and keep prices and availability accurate.

The first version excludes a mobile app, food ordering, cleaning tasks, and nearby-theater suggestions.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Administrator	An employee who manages system information.
Assigned seat	A specific seat reserved for a ticket holder.
Digital ticket	An electronic ticket sent by email.
Digital kiosk	A machine used to buy tickets at the theater.
SRS	Software Requirements Specification.
Showtime	The date and time of a movie showing.
Use case	Steps for completing a task using the system.

1.4 References

- CS 250 Software Requirements Specification Template. San Diego State University, Fall 2023. Provided by Dr. Gus Hanna through Canvas.
- Theater Ticketing System Requirements. Information collected during client interviews at San Diego State University, September 2026.
- IEEE Recommended Practice for Software Requirements Specifications. IEEE Std 830-1998 (R2009), IEEE Computer Society, 1998. Canvas.

- Steve Chenoweth and Chandan Rupakheti. Functional Requirements – Use Cases. Course lecture slides, no date provided. Canvas.
- William Y. Arms. Scenarios and Use Cases. Cornell University, CS 5150 Software Engineering, 2013. Canvas.
- Sridhar Raghavan, Gregory Zelesnik, and Gary Ford. Lecture Notes on Requirements Elicitation. Report CMU/SEI-94-EM-10, Carnegie Mellon University, 1994. Canvas.
- Requirements Elicitation by Brainstorming and Interviewing. Course reading, San Diego State University, 2023. Canvas.
- Group 1. Marvel Electronics and Home Entertainment E-Store Project Software Requirements Specification. Version 4.0, April 16, 2007. Canvas Example.
- Ravi Bandakkanavar. Software Requirements Specification Document with Example. Krazytech, May 28, 2025. Canvas Example.

1.5 Overview

The rest of this document is divided into two main sections. Section 2 explains the system's users, main features, limits, and dependencies. Section 3 covers the interfaces, detailed requirements, use cases, and main objects.

2. General Description

This section explains how the system will be used and what affects how it works.

2.1 Product Perspective

The system will replace the current system, which is slow and has bugs. All theaters will share one database, with information divided by theater. Customer and employee screens will show updated ticket information.

2.2 Product Functions

The system will allow the following:

Customers can:

- Find movies, showtimes, ratings, and reviews.
- Buy tickets and choose seats in deluxe theaters.
- Pay by credit card, PayPal, or Bitcoin.
- Receive printed or digital tickets.
- Create optional accounts and use memberships.
- Use eligible student, military, veteran, or senior discounts.
- Give feedback after a purchase.

Employees can:

- Sell, print, and check tickets.
- Handle refunds in person and help with ticket problems.

Administrators can:

- Manage movies, showtimes, theaters, prices, discounts, and accounts.
- Fix purchase mistakes and view daily sales, ticket prices, and tickets sold per movie.

The system can:

- Block ticket-buying bots and duplicate ticket use.
- Provide a waiting queue during high demand.
- Keep ticket information updated.

2.3 User Characteristics

Customers may have little experience with websites or kiosks. Employees need basic training to handle tickets. Administrators need more training to manage theater and account information.

2.4 General Constraints

- The system must run in a browser.
- It must support English, Spanish, and Swedish.
- Regular theaters use general seating; deluxe theaters use assigned seating.
- Purchases are limited to 20 tickets and five minutes.
- The system must support at least 10,000,000 users at once.
- It must protect customer information and follow theater rules.

2.5 Assumptions and Dependencies

The theater will keep movie, showtime, seat, and price information updated. The system needs internet access, a working database, outside services, kiosks, printers, and scanners. If these change or stop working, some features may be affected.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- Customers shall use the website or theater kiosks in English, Spanish, or Swedish.
- Employees and administrators shall have screens for their ticket and management tasks.

3.1.2 Hardware Interfaces

- The system shall work with kiosks, send ticket details to printers, and read ticket identifiers from scanners.

3.1.3 Software Interfaces

- The system shall connect to the theater database and payment, email, movie review, and currency services.

3.1.4 Communications Interfaces

- The system shall encrypt information exchanged with browsers and outside services.
- Failed connections shall display an error.

3.2 Functional Requirements

3.2.1 Find Movies and Showtimes

- Customers shall search for movies and view showtimes, prices, availability, and reviews.
- The system shall show a message when nothing matches.

3.2.2 Purchase Tickets

- Purchases shall allow up to 20 tickets with a five-minute hold.
- Sales shall open two weeks before showtime and close 10 minutes after.
- The system shall handle discounts, currency conversion, payments, and ticket delivery.
- Payments may be retried before holds expire. Expired holds shall be released.

3.2.3 Manage Customer Accounts

- Optional accounts shall save customer, payment, purchase, and membership information, including loyalty points.
- Each account shall allow one signed-in device and reject invalid logins.

3.2.4 Manage Theater Information

- Administrators shall manage theater information and accounts, correct purchases, and view sales reports.
- Invalid or unauthorized changes shall be rejected.

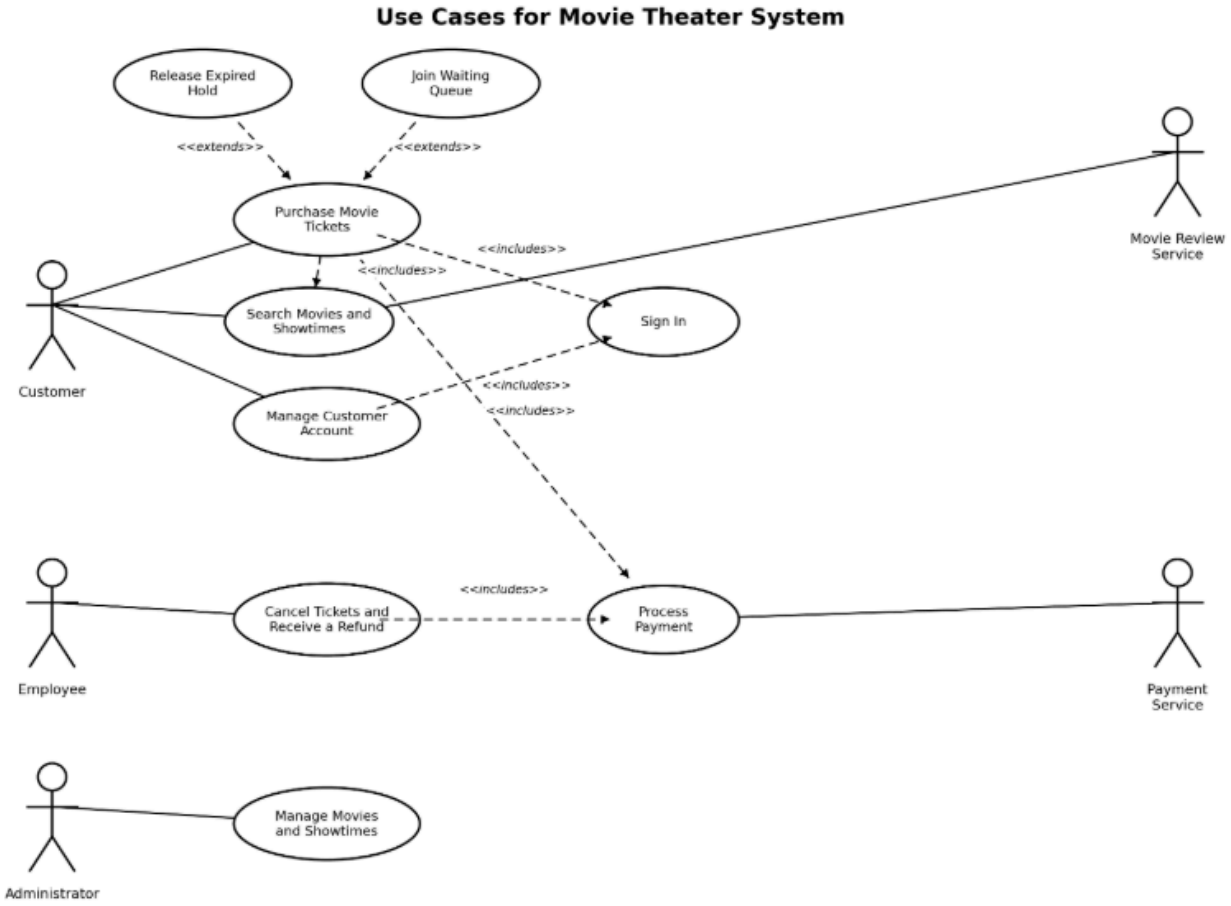
3.2.5 Collect Customer Feedback

- Customers may submit purchase ratings and receive confirmation or an error without affecting their purchase.

3.2.6 Check, Return, and Exchange Tickets

- Employees shall check tickets and handle eligible account-based returns or exchanges before showtime, with refunds handled in person.
- The system shall update availability, reject used tickets, block buying bots, and queue excess requests.

3.3 Use Cases



3.3.1 Purchase Movie Tickets

A customer picks a movie, showtime, and up to 20 tickets. They pick seats for a deluxe theater. The system holds the tickets for five minutes and sends them to the customer after payment goes through. If tickets are unavailable, the customer picks others. They can retry a failed payment before time runs out. Expired holds are released.

3.3.2 Manage Movies and Showtimes

An administrator logs in to add or change movies and showtimes. The system checks and saves the changes, then shows the updated information. Missing details or showtimes that overlap must be fixed first. If the administrator cancels before saving, nothing changes.

3.3.3 Cancel Tickets and Receive a Refund

A customer asks an employee for a refund at the theater before the movie starts. The employee checks that the tickets were bought with an account and qualify for a refund. Once the payment service approves it, the system records the refund and cancels the tickets so they can be sold again. Rejected or failed refunds are not marked complete.

3.4 Classes / Objects

3.4.1 Customer Account

Attributes: Account ID, contact details, saved payments, purchase history, membership, and loyalty points.
Operations: Sign in, update details, and view purchases.

CustomerAccount
accountID: int contactDetails: String savedPayments: String[] purchaseHistory: Ticket[] membership: String loyaltyPoints: int
signIn(): bool updateDetails(String): void viewPurchases(): Ticket[]

3.4.2 Showtime

Attributes: Movie, theater, date, time, price, and availability.
Operations: Display showtimes, hold tickets, and update availability.

Showtime
movie: String theater: String date: String time: String price: double availability: int
displayShowtimes(): void holdTickets(int): bool updateAvailability(int): void

3.4.3 Ticket

Attributes: Ticket ID, showtime, assigned seat when applicable, price, and status.
Operations: Issue, print, check, and cancel tickets.

Ticket
ticketID: int showtime: Showtime

assignedSeat: String [0..1] price: double status: String
issue(): void print(): void check(): bool cancel(): void

3.5 Non-Functional Requirements

3.5.1 Performance

The system shall support at least 10,000,000 concurrent users.

3.5.2 Reliability

The system shall prevent duplicate charges and double booking of assigned seats.

3.5.3 Availability

The system shall provide 99.5% monthly uptime, excluding no more than two hours of scheduled maintenance.

3.5.4 Security

The system shall restrict staff functions to authorized users and prevent customers from accessing other accounts.

3.5.5 Maintainability

Administrators shall update movies, showtimes, prices, and discounts without changing the program code

3.5.6 Portability

The system shall run in a web browser on computers, phones, and theater kiosks without requiring a separate app.

3.6 Inverse Requirements

The system shall not sell more tickets than a showing can hold or sell the same assigned seat twice. It shall not accept used or cancelled tickets.

The first version shall not include a separate mobile app, food ordering, cleaning tasks, or nearby-theater suggestions.

3.7 Design Constraints

The system must run in a browser and support English, Spanish, and Swedish. All 20 San Diego theaters must share the same database. It must work with theater kiosks, ticket printers, scanners, and outside payment services.

Regular theaters use general seating. Deluxe theaters use assigned seats.

3.8 Logical Database Requirements

The database shall store accounts, movies, theaters, showtimes, bookings, tickets, payments, and feedback. Each record shall have a unique ID and link to related records.

Prices shall use exact decimal values and include the currency. Saved payment methods shall use payment service references instead of card numbers.

Purchases and refunds shall update ticket availability. Database checks shall prevent double bookings and duplicate payment records. The client still needs to confirm how long records should be kept.

3.9 Other Requirements

No additional requirements have been identified. Any new requirements will follow the change process in Section 5.

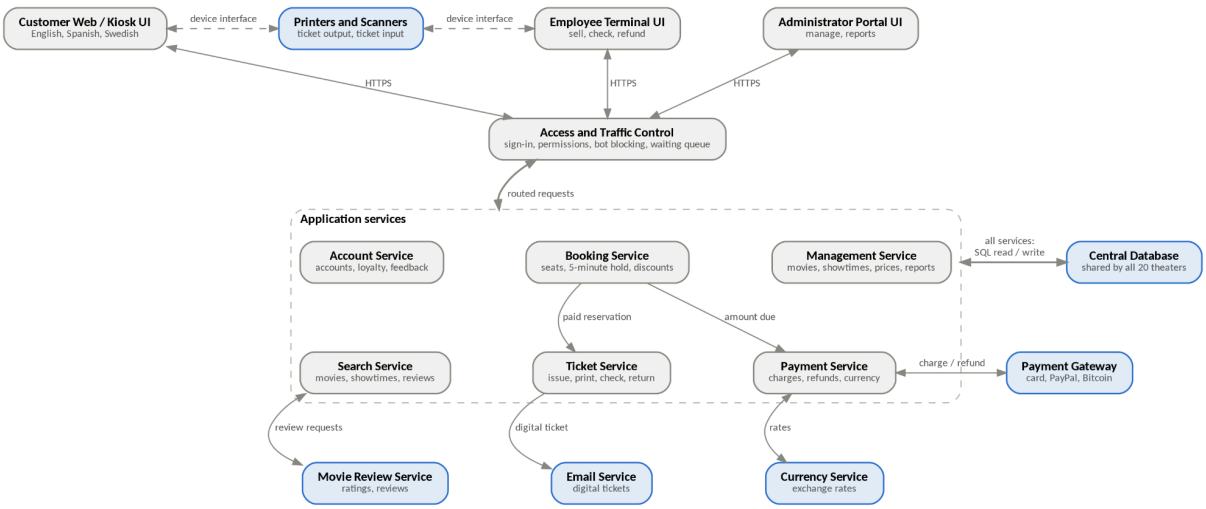
4. Software Design

4.1 System Description

The Movie Theater System is a browser based ticketing system for 20 San Diego theaters that replaces the current slow system and uses one shared database. Customers use either the website or theater kiosks in English, Spanish, or Swedish to find movies and showtimes, choose seats at deluxe theaters and pay by credit card, PayPal, or Bitcoin, and receive printed or digital tickets. Employees sell, check, and refund tickets in person, and administrators manage movies, showtimes, theaters, prices, and sales reports.

4.2 Software Architecture Overview

4.2.1 Software Architecture Diagram



4.2.2 Software Architecture Descriptions

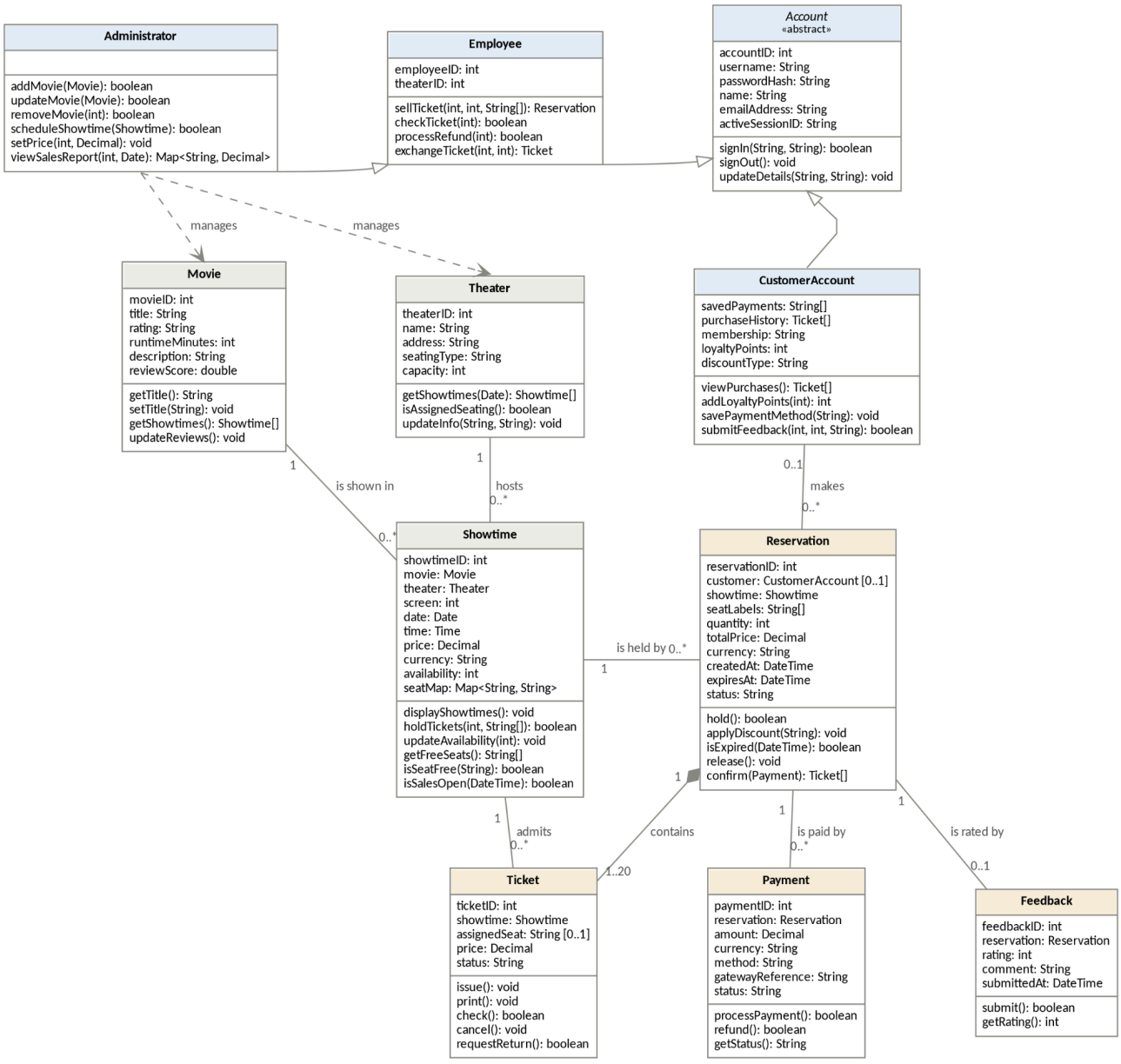
Component	Role
Customer Web / Kiosk UI	Customer screens in English, Spanish, and Swedish.
Employee Terminal UI	Screens to sell, scan, and refund tickets.
Administrator Portal UI	Screens to manage movies, showtimes, prices, and reports.
Printers and Scanners	Print tickets and read ticket IDs.
Access and Traffic Control	Checks sign-in and permissions, blocks bots, and queues extra requests.
Search Service	Finds movies and showtimes with prices, availability, and reviews.
Booking Service	Checks availability, holds seats for up to five minutes, and applies discounts.
Payment Service	Charge payments, converts currency, and handles refunds.
Ticket Service	Issues, prints, checks, and returns tickets.
Account Service	Manages accounts, loyalty points, and feedback.
Management Service	Lets administrators update movies, showtimes, prices, and view reports.
Central Database	One shared database for all records at all 20 theaters.
Payment Gateway (external)	Approve charges and refunds.

Currency Service (external)	Supply correct exchange rates.
Email Service (external)	Send digital tickets.
Movie Review Service (external)	Supplies ratings and reviews.

Connector	Between	Data exchanged
HTTPS	User screens and Access and Traffic Control	Requests and replies
Device interface	Customer and Employee UIs and Printers and Scanners	Ticket details out, ticket IDs in
Routed requests	Access and Traffic Control and the services	Checked requests and results
Amount due / paid reservation	Booking Service to Payment Service and Ticket Service	Reservation ID, total, and currency; reservation ID after approved payment
SQL read / write	Services and Central Database	All records
Outside service calls	Payment Service to Payment Gateway and Currency Service; Ticket Service to Email Service; Search Service to Movie Review Service	Charge and refund requests, exchange rates, digital tickets, and review data

4.3 UML Class Diagram

4.3.1 Class Diagram



4.3.2 Class Diagram Descriptions

The diagram lists each function's inputs and return type. The table explains what each class stores and does.

Class	Attributes and operations
CustomerAccount	Stores login, contact, payment references, membership, and points. Handles account updates and login. Only one session stays active. Administrators control roles.
Movie	Stores titles, runtimes, and reviews. Updates movie details and gets outside reviews.
Theater	Stores theater details, capacity, and seating. Updates this information and provides sales reports.
Showtime	Stores movies, locations, times, prices, and discounts. Finds shows, checks availability, and rejects overlapping schedules.
Reservation	Stores purchase details and hold times. Handles holds, payments, refunds, and exchanges.
Payment	Stores payment records. Handles charges, refunds, and currency conversion. Request IDs prevent repeated charges.
Ticket	Stores ticket details and status. Issues, delivers, and checks tickets. Rejects used, cancelled, or wrong-show tickets.
Feedback	Stores a purchase rating and its time. submit checks the booking and rating, then saves them. Failure does not change the purchase.
Account	Stores login and contact details. Handles signing in, signing out, and updating details.
Employee	Stores employee and theater IDs. Handles ticket sales, checks, refunds, and exchanges.
Administrator	Adds and updates movies, schedules showtimes, sets prices, and views sales reports.

4.4 Development Plan and Timeline

This is a proposed split for Group 5. Each member reviews another member's work and pushes at least one commit. The design document is due October 8, 2026.

Task	Member	Target
Architecture diagram and explanation	Adriane	October 6
UML diagram and class details	Chris	October 7
Development plan and document check	Domenic	October 7
Check and upload the final PDF	All members	October 8

The proposed software build takes 16 weeks, starting when implementation begins. This follows the earlier four-month prototype goal; it is separate from the document deadline.

Weeks	Development task	Lead
1–2	Confirm rules and set up the database.	All
3–5	Build accounts, movie searches, showtimes, and screens.	Adriane
6–9	Build purchases, payments, tickets, and returns.	Chris
10–12	Add administration, reports, memberships, reviews, and feedback.	Domenic
13–16	Test the system and fix problems.	All

The team still needs to confirm discount amounts, membership benefits, and the feedback scale. These are settings, not new requirements.

5. Change Management Process

This section explains how the group updates the SRS when the scope or requirements change.

5.1 Who Can Submit Changes

Any group member can propose a change. The client and the instructor can also ask for changes by email.

5.2 How Changes Are Submitted

Group members submit change requests as GitHub issues. The client and instructor can send them by email. Each request lists the section, the reason, the new wording, and any effect on other requirements.

5.3 Review and Approval

- The group reviews the request at its next meeting, or within three days for urgent changes.
- The group checks the effect on related requirements, diagrams, and the 16-week plan.
- A change is accepted when all three members agree. Scope changes in Section 1.2 also need the client's or instructor's approval.
- Rejected or deferred requests are noted in the GitHub issue with the reason.

5.4 Updating the Document

One member makes the approved edit and updates any affected diagrams. They add a row to the Revision History. A second member checks the update before it is uploaded to GitHub.

6. Test Plan

6.1 Software Design Specification

The software design is unchanged from Assignment 2. The architecture and the UML class diagram are still current. No classes, attributes, operations, or relationships were added, removed, or changed for testing. Figure 1 marks which classes each level of testing covers on the class diagram from Section 4.3.1.

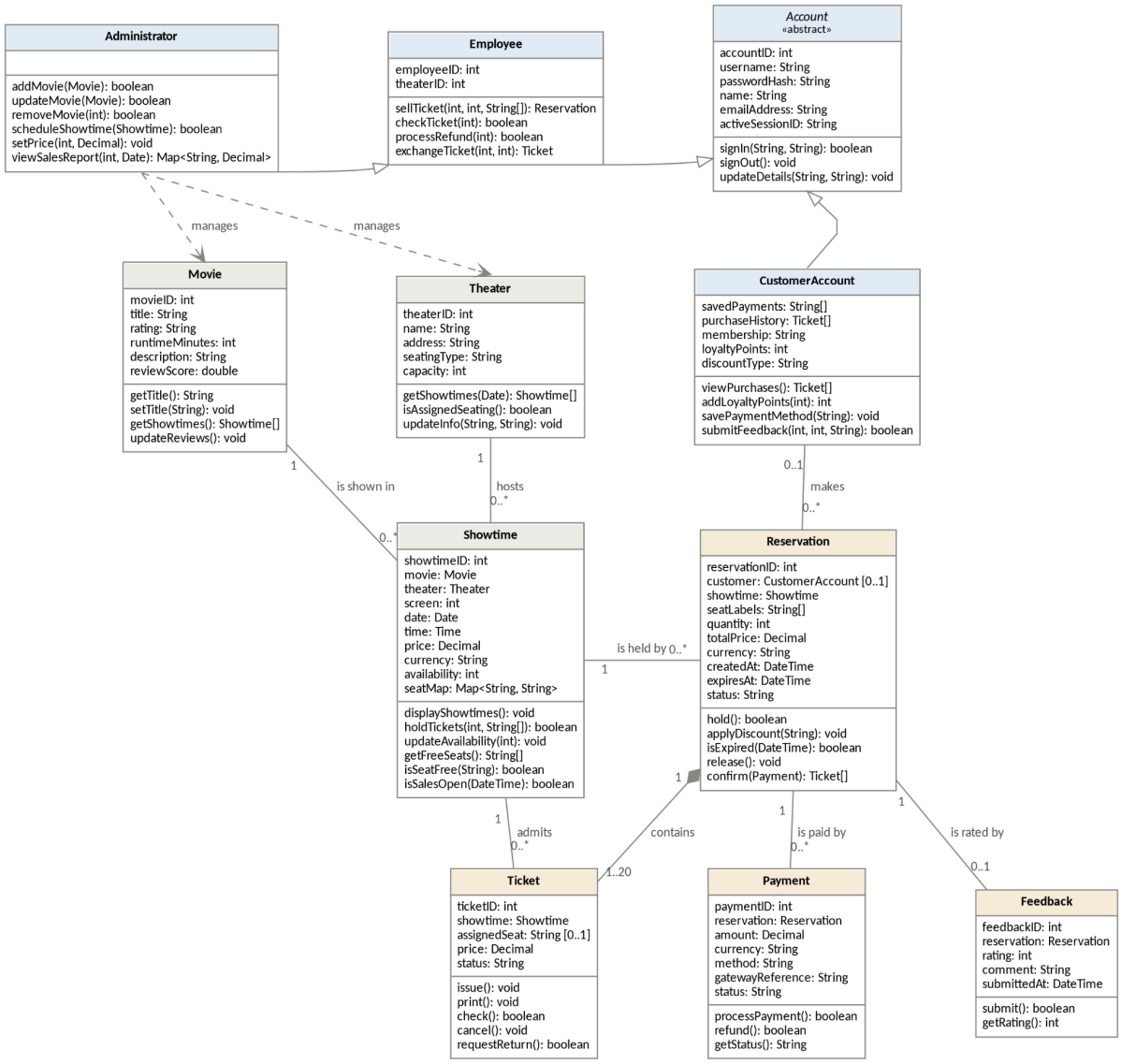


Figure 1. Class diagram (unchanged from Assignment 2)

6.2 Test Approach

Verification asks "are we building the product right?" and checks the implementation against the specification. **Validation** asks "are we building the right product?" and checks against the client's requirements. The unit and integration tests are verification. The system tests walk through the use cases in SRS 3.3 and are validation.

Level	What is tested	Technique	Test Sets
Unit	One method or class	Black-box (partition testing and boundary values) and white-box (branch)	TC-U1 to U4
Integration (functional)	Classes working together	Black-box, interface testing	TC-F1 to F3
System	The complete system	Black-box, use case scenarios, load testing	TC-S1 to S3

Black-box tests come from the specification. For each feature, the test sets list the relevant inputs, split them into groups that should behave alike, and test values at and next to the boundaries of each group. White-box tests come from the code. The unit tests aim to run every branch of the method under test.

6.3 Test Sets

Each test set lists the features tested, the inputs, the specific test cases with expected results, and the failures it can catch. Six out of ten test sets are described here, two for each level.

6.3.1 Unit tests

TC-U1: Sales window, Showtime.isSalesOpen()

Feature: Sales open 2 weeks before showtime and close 10 minutes after it

Inputs: The current time, test both boundaries for a showtime on 2026-10-20 at 19:00.

Test cases: One second before opening (2026-10-06 18:59:59) gives false. Opening (19:00:00) gives true. One second before closing (2026-10-20 19:09:59) gives true. Closing (19:10:00) gives false. A null time gives an error.

Catches: off-by-one mistakes.

TC-U4: Ticket scan, Ticket.check()

Feature: Reject used, cancelled, and/or wrong-show tickets.

Inputs: The ticket status (VALID, USED, CANCELLED) and whether the scan is at the right showtime. These four cases are also the four branches of check().

Test cases: A VALID ticket at the right showtime gives true and becomes USED. A USED ticket gives false. A CANCELLED ticket gives false. A VALID ticket scanned at a different showtime gives false and stays VALID. **Catches:** a ticket used twice, entry with a refunded ticket.

6.3.2 Integration (functional) tests

TC-F1: Double booking, Reservation and Showtime

Feature: One assigned seat in a deluxe theater can only be held by one reservation. Tests call from Reservation.hold() to Showtime.isSeatFree() and holdTickets().

Inputs: The same seat or different seats, requested one after another at the same moment.

Test cases: If A and B hold seat D5 one after the other, A succeeds and B fails with "seat unavailable". If A and B hold D5 at the same moment, exactly one succeeds and the database has one row for D5. If A holds D5 and B holds D6, both succeed.

Catches: Race conditions and selling a single seat twice.

TC-F2: Payment retry, Reservation, Payment, and Ticket

Feature: A failed payment can be retried before the hold expires and repeated requests never charge multiple times, and expired holds are released.

Inputs: The payment answer(declined or approved), a new or repeated request ID, and a time for the five-minute hold.

Test cases: For a 2 ticket reservation, a declined payment at 1:00 leaves it HELD with no tickets. An approved repayment at 2:00 makes it CONFIRMED with 2 VALID tickets. Sending the same request ID again doesn't charge again. A payment at 5:01 is refused, the reservation is EXPIRED, and the seats are FREE.

Catches: A charge with no ticket, a ticket with no charge, double charges.

6.3.3 System tests

TC-S1: Buy and use tickets, Use Case 3.3.1

Feature: The full customer path runs in a real browser against the deployed system.

Inputs: Search for a movie, choose a deluxe showtime, pick 3 seats, apply a student discount, pay by credit card, open the emailed ticket, and scan it at the employee terminal, all translated in English, Spanish, or Swedish.

Test cases: The price includes the discount, the tickets are the chosen seats, there is one charge, the email arrives, the first scan is accepted, second scan is rejected, and the seats no longer are available. **Catches:** Wrong prices or seats, missing emails, untranslated screens.

TC-S2: Heavy demand, SRS 3.5.1 and 3.5.2

Feature: No overselling or double charging when many people buy at once, extra requests wait in a queue.

Inputs: A showtime with 10 seats left and 1,000 users attempting to purchase a ticket at the same moment. Repeated at 10,000 and 100,000 users, plus a bot sending 200 requests per second. The SRS asks for 10,000,000 users, loads must be scaled down.

Test cases: Exactly 10 tickets and 10 payments exist, no seat is sold twice, other users are queued/told tickets are sold out, no server errors, and the bot is blocked.

Catches: Overselling, double charges, crashes, bots getting through.

6.4 Test Cases and Repository

The ten test cases are located in testCases.xlsx with a traceability sheet linking each test to the SRS. The Actual Result and Pass/Fail columns are filled later when tests are run.

GitHub repository: https://github.com/cnguyen1939/cs250-movietheater